Progetto Criacci-Sambuco-Sereni

Joseph Criacci, Matilde Sambuco, Leonardo Sereni

***Abstract*—Obiettivo di questo progetto è comprendere e apprezzare le funzionalità di una scheda FPGA programmata in VHDL per l'implementazione di sistemi embedded.**
Nello specifico è stato realizzato un sistema con funzione di cronometro, timer e orologio prendendo spunto da un classico orologio digitale i cui pulsanti permettono di passare da una modalità all'altra e di configurare la modalità scelta.
Sistemi di questo tipo trovano oggi applicazione nel campo della robotica e sono integrati in *smartwatch*, dispositivi *smart* per la domotica e non solo.
Il sistema è implementato come una macchina a stati finiti. L'impostazione gerarchica del progetto permette di gestire in modo efficiente le varie funzionalità del sistema e vuole riflettere la *reusability*, tipica dello sviluppo di sistemi embedded.

***Index Terms*—buzzer, *clock* divider, macchina a stati finiti, sette segmenti**

I. Introduzione

Il rapido progresso dell'elettronica digitale nelle applicazioni commerciali ha trasformato la società moderna, consentendo lo sviluppo di dispositivi di consumo sempre più sofisticati. Dai microcontrollori integrati negli elettrodomestici ai complessi sistemi basati su FPGA, la tecnologia digitale è diventata fondamentale per la progettazione elettronica contemporanea.

L'obiettivo di questo progetto è la realizzazione, tramite utilizzo di FPGA Basys 3, di un orologio digitale, in grado di visualizzare l'ora corrente, cronometrare eventi e impostare allarmi. L'orologio deve essere in grado di visualizzare l'ora in formato 24h, con precisione di un minuto, e deve essere possibile impostare l'ora corrente tramite appositi pulsanti. Inoltre, deve essere possibile avviare, fermare e resettare un cronometro, che deve essere visualizzato sullo stesso *display* dell'ora corrente, con precisione fino al centesimo di secondo. Infine, deve essere possibile impostare un'allarme che suoni dopo un lasso di tempo stabilito dall'utente.

A. Interfacce Utente

Il progetto mira a ricreare un classico orologio digitale da polso, utilizzando un *display* a sette segmenti a quattro cifre per la visualizzazione dell'ora, del cronometro e del timer. L'interfaccia utente sarà costituita da pulsanti fisici per l'impostazione dell'ora, l'avvio/stop del cronometro e la configurazione del timer. L'uso di un *display* a sette segmenti consente una visualizzazione chiara e intuitiva delle informazioni temporali, mentre i pulsanti integrati alla scheda Basys 3 offrono un metodo semplice per interagire con il sistema.

Viene inoltre integrato il collegamento ad un *buzzer* esterno, che emette un segnale acustico allo scadere del timer, fornendo un *feedback* immediato all'utente.

B. Funzionamento del Dispositivo

Il sistema si inizializza in modalità cronometro, dando la possibilità di far partire e interrompere la misura dello scorrere del tempo un numero indefinito di volte tramite il pulsante *RIGHT* dell'FPGA. Premere il pulsante di *reset* (*DOWN*), resetterà il cronometro, riportando automaticamente il valore del cronometro a zero.
La funzione timer prevede la possibilità di impostare un conto alla rovescia, utilizzando i bottoni *UP* e *CENTER* della scheda. Questo potrà essere iniziato, interrotto o ripreso in qualunque momento, tramite il pulsante *RIGHT*, ma sarà necessario resettarlo, tramite il pulsante *DOWN*, prima di poterlo reimpostare. Al termine del conto alla rovescia, il *display* e tutti i led della scheda dovranno illuminarsi e, se collegato, il *buzzer* dovrà riprodurre un suono di allarme.
L'orologio integrato può essere impostato a piacimento, sempre tramite i pulsanti *UP* e *CENTER*, ed esso rimarrà attivo anche se il *display* sarà impegnato in un'altra funzione, fintanto che la scheda rimarrà alimentata.
Infine, dovrà sempre essere possibile passare da una modalità all'altra della scheda premendo il tasto *LEFT*, alternando in ordine le modalità di cronometro, timer e sveglia.

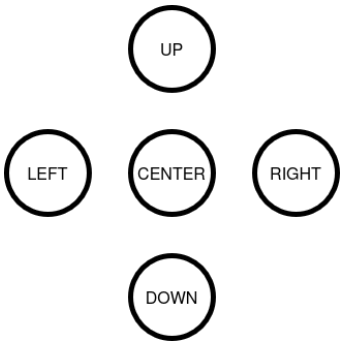

Fig. 1. Layout pulsanti scheda Basys 3

II. Struttura del Progetto

Il sistema è stato progettato come una macchina a stati finiti (FSM), con stati distinti per la visualizzazione dell'ora, la gestione del cronometro e la configurazione dell'allarme. Ogni stato gestisce specifiche funzionalità e transizioni basate sugli *input* dell'utente. Questa struttura modulare consente una facile espansione e manutenzione del codice, facilitando l'aggiunta di nuove funzionalità in futuro. Il progetto è stato suddiviso in moduli funzionali per facilitare lo sviluppo e la gestione del codice, così come è osservabile in figura 2:

Fig. 2. Struttura progetto VHDL

I principali moduli includono:

- **clk_wiz_0**: Modulo per la generazione del segnale di *clock* di sistema.
- **display_7seg**: Modulo per la visualizzazione sul *display* a sette segmenti.
- **btn_listener**: Modulo per la gestione dei bottoni di *input*.
- **state_changer**: Modulo per la logica della macchina a stati finiti (FSM).
- **state_handler**: Modulo per la gestione dell'orologio, cronometro e allarme.

A. Modulo: project_main

Il modulo **project_main** funge da punto di ingresso principale del progetto, integrando tutti i moduli funzionali e coordinando le loro interazioni. Questo è il modulo *top-level* che si interfaccia direttamente con i segnali di *input* e *output* della scheda Basys 3, come il *clock* a 100 MHz generato dalla scheda, i bottoni fisici, il *display* a sette segmenti e il *buzzer* esterno. Le porte associate a questo modulo sono le seguenti:

- **clk_in1**: Segnale di *clock* di ingresso a 100 MHz generata dalla scheda Basys 3.
- **reset**: Segnale associato allo switch **SW0** della scheda, utilizzato per bloccare la generazione del *clock* di sistema.
- **btnU, btnD, btnL, btnR, btnC**: Segnali associati ai bottoni fisici della scheda (Up, Down, Left, Right, Center).
- **seg**: Segnali di uscita per i segmenti del *display* a sette segmenti.
- **dp**: Segnale di uscita per il punto decimale del *display* a sette segmenti.
- **an**: Segnali di uscita per l'attivazione dei singoli *display* a sette segmenti.
- **led**: Segnali di uscita per i LED integrati nella scheda Basys 3
- **buzzer_out**: Segnale di uscita per il controllo del *buzzer* esterno.

Tale interconnessione tra i segnali logici e fisici è possibile grazie all'uso di un file di *constraints* (**Basys3_Master.xdc**), che mappa i segnali logici del progetto ai *pin* fisici della scheda FPGA.

All'interno di questo modulo, vengono istanziati e collegati tutti gli altri moduli del progetto, garantendo una comunicazione fluida tra di essi. In figura 3 possiamo osservare il diagramma RTL del modulo **project_main**.

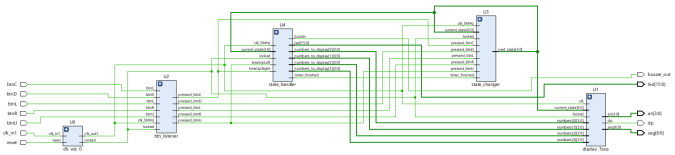

Fig. 3. Diagramma RTL del modulo *project_main*

B. Modulo: clk_wiz_0

Il modulo **clk_wiz_0** è responsabile della generazione di un segnale di *clock* di sistema a 5 MHz, partendo dal *clock* di ingresso a 100 MHz fornito dalla scheda Basys 3. Questo segnale di *clock* è fondamentale per il funzionamento sincronizzato di tutti i moduli del progetto, garantendo che le operazioni temporali siano eseguite con precisione. Si è scelto di utilizzare l'IP *Clocking Wizard* di Vivado per creare questo modulo, in quanto offre un'interfaccia intuitiva per configurare i parametri del *clock*, come la frequenza di uscita desiderata e le opzioni di *jitter*. L'uso di questo IP semplifica notevolmente il processo di generazione del *clock*, riducendo il rischio di errori di configurazione e migliorando l'affidabilità complessiva del sistema.

Il modulo **clk_wiz_0** riceve in *input* il segnale di *clock* a 100 MHz e il segnale di reset, e produce in *output* il segnale di *clock* a 5 MHz, che viene poi distribuito agli altri moduli del progetto per garantire un funzionamento sincronizzato e preciso, e un segnale noto come **locked**. Quest'ultimo segnale indica che il *clock* generato è stabile e pronto per essere utilizzato, assicurando che tutti i moduli del sistema operino correttamente sin dall'inizio.

C. Modulo: display_7seg

Il modulo **display_7seg** si occupa della gestione dei *display* a sette segmenti integrati nella scheda Basys 3. Questo modulo riceve in *input* un segnale di *clock* a 5 MHz e i dati da visualizzare, e si occupa di convertire questi dati nel formato appropriato per il *display*. La gestione del *multiplexing* dei *display* è fondamentale per garantire una visualizzazione chiara e leggibile delle informazioni temporali, come l'ora corrente, il cronometro e il timer.

Le porte definite sono le seguenti:

- **clk**: Segnale di ingresso del *clock* a 5 MHz.
- **locked**: Segnale di indica la stabilità del *clock* generato.
- **reset**: Segnale di ingresso di *reset* associato allo *switch* **SW0** della scheda.
- **numbers**: Segnale di ingresso di tipo *digit_array* (*user defined*) costituito dalle 4 cifre decimali da visualizzare sul *display*.
- **current_state**: Segnale di ingresso di tipo *state_type* (*user defined*) che indica lo stato attuale della macchina a stati finiti, utilizzato per determinare quali segmenti del *display* devono essere attivati.
- **seg**: Segnali di uscita per la codifica binaria dei numeri da rappresentare sul *display* a sette segmenti.
- **an**: Segnali di uscita per l'attivazione dei singoli *display* a sette segmenti.
- **dp**: Segnale di uscita per il punto decimale del *display* a sette segmenti.

Inoltre, al suo interno è presente un modulo secondario chiamato **clk_1kHz_divider**, il cui scopo è quello di generare un segnale di *clock* a 1 kHz a partire dal *clock* di ingresso a 5 MHz. Il segnale di *clock* a 1 kHz viene utilizzato per il *multiplexing* dei *display*, permettendo di aggiornare i segmenti in modo rapido e fluido, evitando problemi di "*flickering*", causato da una frequenza di aggiornamento troppo bassa, e problemi di leggibilità dovuti a una frequenza di aggiornamento troppo alta.

D. Modulo: btn_listener

Il modulo **btn_listener** è progettato per gestire gli *input* provenienti dai cinque bottoni fisici presenti sulla scheda Basys 3. Il modulo riceve in *input* i segnali grezzi dei bottoni (**btnU, btnD, btnL, btnR, btnC**) e il segnale di *clock* a 5 MHz generato dal modulo **clk_wiz_0**. Ogni qual volta il valore di uno dei bottoni cambia da basso ad alto (pressione del bottone), il modulo genera un impulso di durata pari ad un ciclo di *clock* a 5 MHz (200 ns) sul corrispondente segnale di uscita (**pressed_btnU, pressed_btnD, pressed_btnL, pressed_btnR, pressed_btnC**). Questo approccio consente di evitare problemi di rimbalzo (*debouncing*) e garantisce che ogni pressione del bottone venga riconosciuta in modo affidabile dal sistema.

Le porte definite per questo modulo sono le seguenti:

- **clk_5MHz**: Segnale di ingresso del *clock* a 5 MHz.
- **locked**: Segnale di ingresso che indica la stabilità del *clock* generato.
- **btnU, btnD, btnL, btnR, btnC**: Segnali di ingresso associati ai bottoni fisici della scheda (Up, Down, Left, Right, Center).
- **pressed_btnU, pressed_btnD, pressed_btnL, pressed_btnR, pressed_btnC**: Segnali di uscita che generano un impulso di durata pari a un ciclo di *clock* quando il corrispondente bottone viene premuto.

E. Modulo: state_changer

Il modulo **state_changer** implementa la logica della macchina a stati finiti (FSM) che governa il comportamento complessivo del sistema, composta dai seguenti stati:

- **CRON_RESET**: Stato in cui il cronometro viene resettato al valore iniziale.
- **CRON_COUNT**: Stato in cui il cronometro è attivo e incrementa il tempo trascorso.
- **CRON_STOP**: Stato in cui il cronometro è fermo e mantiene il tempo attuale.
- **TIMER_RESET**: Stato in cui il timer viene resettato al valore iniziale.
- **TIMER_SET**: Stato in cui l'utente può impostare il valore del timer.
- **TIMER_COUNTDOWN**: Stato in cui il timer è attivo e decrementa il tempo rimanente.
- **TIMER_STOP**: Stato in cui il timer è fermo e mantiene il tempo attuale.
- **TIMER_BEEP**: Stato raggiunto quanto il timer raggiunge lo zero.
- **WATCH**: Stato in cui è possibile impostare e visualizzare l'ora corrente.

Questo modulo riceve in *input* **current_state**, rappresentante lo stato corrente del sistema, e gli impulsi generati dal modulo **btn_listener**, e determina lo stato futuro del sistema, associato al segnale di uscita del modulo, **next_state**.

Ogni stato della FSM è associato a specifiche funzionalità e comportamenti, e il modulo **state_changer** garantisce che il sistema risponda correttamente agli *input* dell'utente, facilitando un'interazione intuitiva e fluida. In figura 4 è possibile osservare il diagramma degli stati implementato nel modulo **state_changer**.

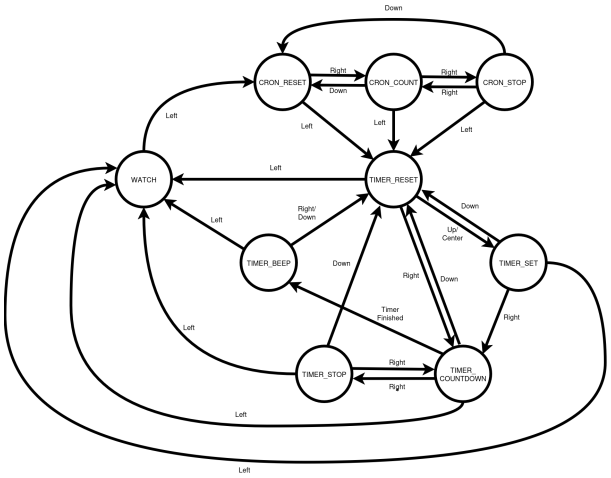

Fig. 4. FSM del modulo *state_changer*

F. Modulo: state_handler

Il modulo **state_handler** è responsabile della gestione delle funzionalità specifiche associate a ciascuno stato della macchina a stati finiti (FSM) implementata nel modulo **state_changer**. Questo modulo si occupa di eseguire le operazioni necessarie la gestione del cronometro, del timer, e l'aggiornamento dell'ora corrente, in base allo stato attivo del sistema.

Le porte definite per questo modulo sono le seguenti:

- **current_state**: Segnale di ingresso di tipo *state_type* (*user defined*) che indica lo stato attuale della macchina a stati finiti.
- **clk_5MHz**: Segnale di ingresso del *clock* a 5 MHz.
- **locked**: Segnale di ingresso che indica la stabilità del *clock* generato.
- **timeUpLeft, timeUpRight**: Segnali di ingresso di tipo *std_logic* che rappresentano i bottoni di *input* per l'impostazione del timer e dell'ora corrente.
- **numbers_to_display**: Segnale di uscita di tipo *digit_array* (*user defined*) costituito dalle 4 cifre decimali da visualizzare sul *display* a sette segmenti.
- **timer_finished**: Segnale di uscita di tipo *std_logic* che indica quando il timer ha raggiunto lo zero.
- **led**: Segnali di uscita per i LED integrati nella scheda Basys 3.
- **buzzer**: Segnale di uscita di tipo *std_logic* che abilita il *buzzer* esterno quando l'allarme viene attivato.

Il modulo contiene la logica necessaria per gestire le operazioni temporali in base allo stato corrente del sistema. In particolare, vengono utilizzate variabili interne per tenere traccia del tempo trascorso nel cronometro, del tempo rimanente nel timer, e dell'ora corrente. Il modulo si occupa di aggiornare questi valori in modo appropriato in base agli *input* dell'utente e alle transizioni di stato, e di elaborare correttamente tali valori per abilitare i led, e popolare l'array **numbers_to_display**, che viene poi inviato al modulo **display_7seg** per la visualizzazione sul *display* a sette segmenti.

Inoltre, il modulo utilizza due sottomoduli di divisione del *clock*, **clk_100Hz_divider** e **clk_watch_divider**, per ottenere segnali di *clock* a frequenze più basse necessari per le operazioni temporali. Il sotto-modulo **clk_watch_divider** genera un segnale di *clock* di periodo pari a 1 minuto (1/60 Hz), utilizzato per incrementare i valori dell'ora corrente. Questo segnale è sempre attivo, garantendo che l'ora venga aggiornata regolarmente. Il sotto-modulo **clk_100Hz_divider** genera un segnale di *clock* a 100 Hz, che viene abilitato solo durante gli stati CRON_COUNT e TIMER_COUNTDOWN. Questo approccio consente di garantire conteggi precisi e accurati per il cronometro e il timer, evitando errori di temporizzazione durante le operazioni di conteggio.

Viene inoltre utilizzato un sotto-modulo chiamato **music_player**, che si occupa di generare un'onda quadra da inviare al *buzzer* esterno quando l'allarme viene attivato.

G. Moduli: clk_1kHz_divider, clk_100Hz_divider e clk_watch_divider

Il progetto utilizza svariati moduli di divisione del *clock* per generare segnali di *clock* a frequenze più basse, necessarie per il corretto funzionamento delle varie funzionalità del sistema.

Il funzionamento di tali moduli si basa su un semplice contatore che incrementa il suo valore ad ogni ciclo del *clock* di ingresso. Quando il contatore raggiunge un valore predefinito, esso si resetta a zero e genera un cambiamento di stato sul segnale di uscita del *clock* diviso. Questo processo continua ciclicamente, producendo un segnale di *clock* a una frequenza inferiore rispetto al *clock* di ingresso.

Per calcolare il numero di cicli necessari per ottenere la frequenza desiderata, è sufficiente invertire il valore di uscita ogni $\frac{f_{clk,ingresso}}{2 \cdot f_{clk,desiderata}}$ cicli del *clock* di ingresso. Ad esempio, per ottenere un *clock* a 1 kHz da un *clock* di ingresso a 5 MHz, il contatore deve raggiungere 2500 cicli prima di invertire lo stato del segnale di uscita.

H. Modulo: music_player

Il modulo **music_player** è progettato per generare un'onda quadra, che viene utilizzata per attivare il *buzzer* esterno collegato alla scheda Basys 3. Questo modulo è fondamentale per la funzionalità di allarme del sistema, poiché consente di emettere un segnale acustico allo scadere del timer.

Il mdulo riceve in *input* il segnale di *clock* a 5 MHz, il segnale **locked** per garantire la stabilità del *clock*, e un segnale di abilitazione (**enable**).

Quando il segnale di abilitazione è attivo, il modulo genera un'onda quadra a una frequenza di oscillazione predefinita che viene inviata al *buzzer* esterno tramite il segnale di uscita **buzzer_out**. Questo permette al sistema di emettere un suono distintivo quando l'allarme è attivato, fornendo un *feedback* immediato all'utente.

III. Testbench e Simulazioni

Il modello del progetto è stato verificato attraverso l'uso di **testbench VHDL**, che simulano il comportamento del sistema in diverse condizioni operative.

Sono stati creati due testbench principali, uno per analizzare il comportamento del sistema durante la gestione del cronometro, e un altro per verificare il corretto funzionamento del timer e dell'allarme. Questi testbench includono la generazione di segnali di *clock*, l'attivazione dei bottoni di *input*, e la verifica delle uscite del sistema, come la visualizzazione sul *display* a sette segmenti e l'attivazione del *buzzer*.

Al fine di rendere più interpretabili le simulazioni, si è deciso di non utilizzare il modulo **project_main** come unità sotto test (UUT) nei testbench, bensì un modulo secondario **project_main_tb**, dal comportamento identico a quello del modulo principale, ma con l'aggiunta di 2 segnali di *output*: **state**, per monitorare lo stato corrente della macchina a stati finiti, e **display_numbers**, per visualizzare i numeri attualmente mostrati sul *display* a sette segmenti. Questi segnali facilitano l'analisi del comportamento del sistema durante le simulazioni, permettendo di verificare che le transizioni di stato e le visualizzazioni siano corrette in risposta agli *input* simulati.

A. Testbench: Cronometro

La prima verifica nella simulazione del cronometro consiste nel monitoraggio delle transizioni di stato della macchina a stati finiti. Dopo un'attesa di 10 ms per stabilizzare il sistema, vengono simulati una serie di impulsi sui bottoni per passare da uno stato all'altro. Le transizioni osservate sono coerenti con il diagramma degli stati, confermando che la logica di transizione funziona correttamente (Figura 5).

Successivamente (all'istante 40 ms), viene verificata la precisione del conteggio del cronometro. Il cronometro viene

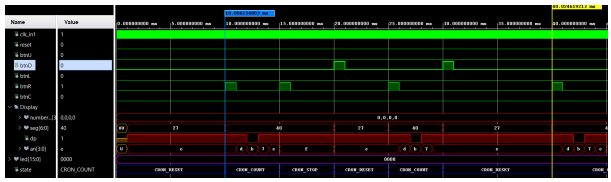

Fig. 5. Simulazione: transizioni di stato

avviato e lasciato funzionare per 13 ms, dopodiché viene fermato per 5 ms, e poi riavviato per altri 7 ms. Al termine della simulazione, il valore visualizzato sul *display* corrisponde esattamente al tempo totale trascorso (20 ms), confermando la precisione del conteggio del cronometro (Figura 6).

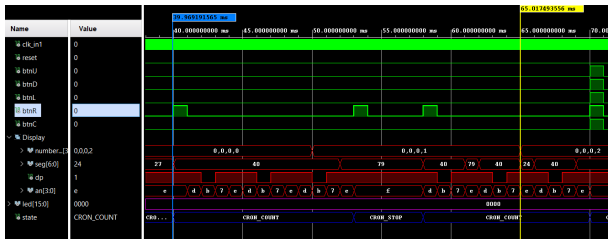

Fig. 6. Simulazione: verifica della precisione del conteggio

Inolte, è stato testato il funzionamento dei bottoni in due casi particolari. Nel primo vengono premuti tutti i bottoni contemporaneamente. La simulazione mostra che il sistema gestisce correttamente questa situazione, eseguendo la prima istruzione dell'*if* dello stato corrente: dallo stato di CRON_COUNT si passa a CRON_STOP dando priorità all'impulso corrispondente al pulsante R.

Nel secondo caso, viene tenuto premuto il pulsante R per 20 ms, determinando il passaggio allo stato da CRON_RESET A CRON_COUNT, e dopo i primi 10 ms viene premuto il pulsante D, passando di nuovo allo stato di CRON_RESET. Poichè i passaggi di stato sono determinati dal fronte di salita degli impulsi generati dai pulsanti, il sistema gestisce correttamente questa situazione poichè i fronti di salita delle due transisioni non sono allineati. (Figura 7).

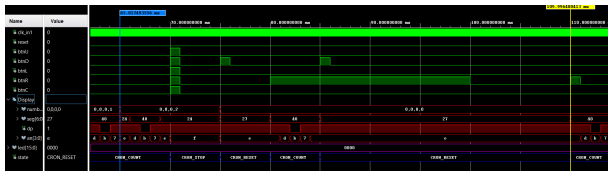

Fig. 7. Simulazione: verifica del funzionamento dei bottoni

Infine, viene verificato il corretto aggiornamento del conteggio del cronometro. Dopo aver avviato il cronomento, e lasciato funzionare per 1 secondo, le cifre visualizzate sul *display* vengono correttamente convertite da centesimi di secondo a secondi, passando da raffigurare ”00:99” a raffigurare ”01:00” (Figura 8).

B. Testbench: Timer e Allarme

La prima verifica per il modulo del timer consiste nel testare le transisizioni di stato in risposta agli impulsi simulati dai

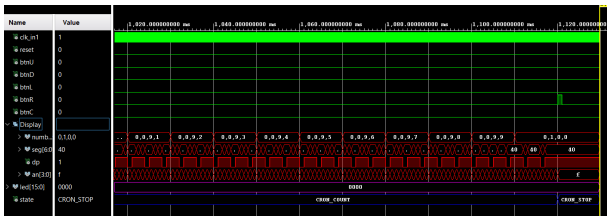

Fig. 8. Simulazione: verifica del corretto aggiornamento del conteggio

bottoni. Dopo un'attesa di 10 ms per stabilizzare il sistema, le transizioni osservate sono coerenti con il diagramma degli stati (Figura 9).

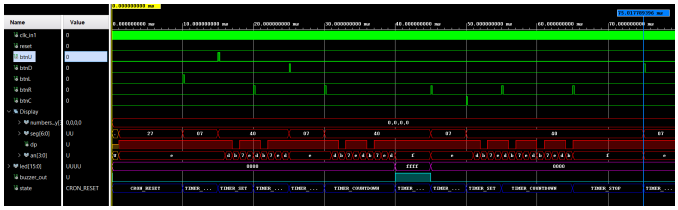

Fig. 9. Simulazione: transizioni di stato

Inoltre si è testato il corretto passaggio allo stato di WATCH (Figura 10). Per questo modulo non sono state fatte ulteriori veriche in quanto la logica di conteggio e visualizzazione dell'ora è identica a quella dei moduli già testati.

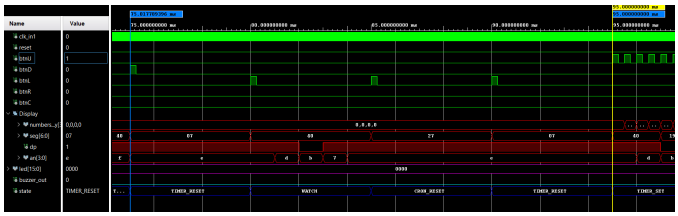

Fig. 10. Simulazione: Transizione orologio

Come ultima verifica in fase di simulazione, è stato testato il corretto aggiornamento dei valori timer. In particolare, dopo aver premuto il pulsante U, dedicato all'incremento dei minuti, per 101 volte e il pulsante C, destinato all'incremento dei minuti, per 62 volte, l'uscita simulata del *display* mostra correttamente il valore impostato di 01:02 (Figura 11).

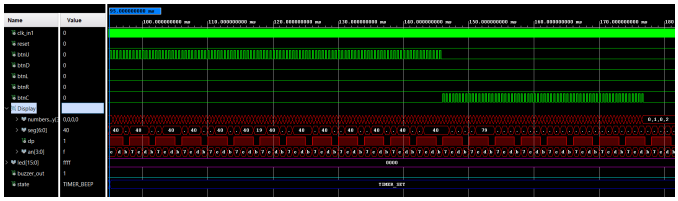

Fig. 11. Simulazione: verifica del funzionamento dei bottoni

Questo è coerente con la logica implementata: dopo 100 pressioni del pulsante U, i minuti ritornano a 00 e con l'ultimo impulso si impostano i minuti pari a 1; allo stesso modo, dopo 60 pressioni del pulsante C, i secondi ritornano a 00 e con gli ultimi 2 impulsi vengono incrementati di 2.

IV. Implementazione e Risultati

Il progetto è stato realizzato con una scheda FPGA **Basys 3** di produzione della Digilent e programmato in linguaggio VHDL utilizzando l'ambiete di sviluppo **Vivado Design Suite**.

A. Collegamenti circuitali

La scheda FPGA viene alimentata tramite un cavo USB 2.0, collegato al computer. Oltre a fornire l'alimentazione, il cavo USB permette anche il trasferimento della *bitstream*, necessario per configurare e programmare l'FPGA.

Viene utilizzato un *buzzer* esterno, collegato alla scheda Basys 3 tramite un transistor NPN (2N2222A), che funge da interruttore controllato dal segnale logico generato dal modulo **music_player**.

I collegamenti tra la scheda Basys 3, il *buzzer* esterno e il transistor sono i seguenti:

- *Pin* **JB6** (VCC) della scheda collegato al terminale collettore del transistor NPN
- *Pin* **JB1** della scheda collegato al terminale base del transistor NPN tramite una resistenza da 1 kΩ
- Terminale emettitore del transistor NPN collegato al terminale positivo del *buzzer*
- Il terminale negativo del *buzzer* collegato al *Pin* **JB5** (GND) della scheda

Il layout del collegamento è illustrato in figura 12.

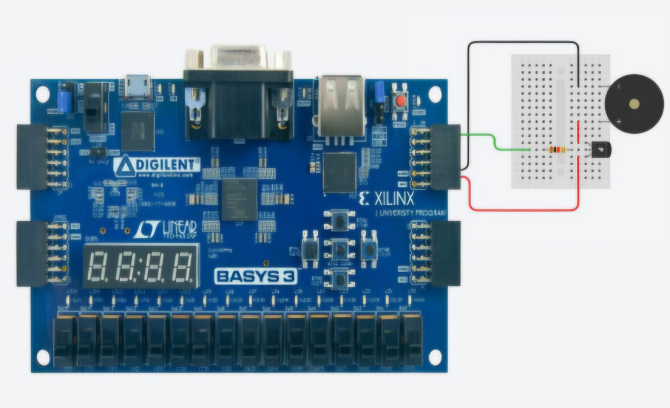

Fig. 12. Collegamento del *buzzer* esterno alla scheda Basys 3

B. Utilizzo delle Risorse

In figura 13 sono riportati i dati relativi all'utilizzo delle **risorse** della scheda Basys 3 per la realizzazione del progetto, in figura 14 sono invece rappresentati i dati relativi al **consumo di potenza** a seguito dell'implementazione, per un consumo totale di circa 0.17 W.

In percentuale, le risorse maggiormente sfruttate risultano essere le **IO** (34% di quelle disponibili), seguite dalle **LUT** (23% di quelle totali) e le risorse dedicate a **Mixed-Mode *Clock* Manager** (20%).

I consumi di potenza del progetto si dividono tra:

- **Potenza statica** (42%): tale consumo dipende dalle perdite intrinsiche della scheda FPGA in assenza di commutazioni o transisizioni di segnale.

Resource	Utilization	Available	Utilization...
LUT	4761	20800	22.89
FF	330	41600	0.79
IO	36	106	33.96
BUFG	2	32	6.25
MMCM	1	5	20.00

Fig. 13. Utilizzo delle risorse della scheda Basys 3

- **Potenza dinamica** (58%): il contributo principale (88%) deriva da risorse **Mixed-Mode *Clock* Manager (MMCM)** che permettono di generare *clock* a frequenze ridotte a partire dal *clock* di ingresso a 100 MHz della scheda, proprio come il *Clocking Wizard* utilizzato nel progetto per realizzare i divisori di frequenza necessari al conteggio.

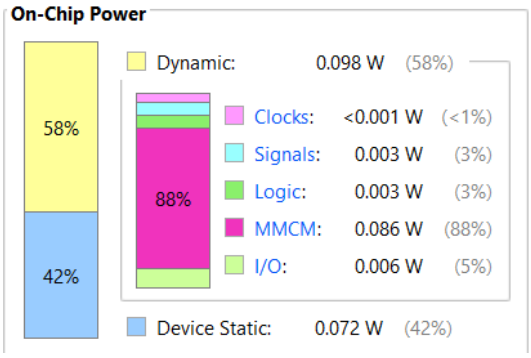

Fig. 14. Dettagli sul consumo di potenza

V. Conclusioni e Sviluppi Futuri

Il dispositivo realizzato in VHDL e implementato su scheda FPGA è in grado di operare come cronometro, orologio e timer, mantenendo sempre una precisione nell'ordine dei centisecondi. Il numero estensivo di simulazioni e di test sperimentali ha permesso di verificarne il corretto funzionamento in tutte le modalità proposte. L'utilizzo di *clock* a diverse frequenze consente al dispositvo di avere un consumo di potenza minimo, riducendo il numero di operazioni svolte dalla scheda senza alcuna perdita di precisione valutabile.
Infine, l'impiego di un sistema FSM permette una gestione modulare e ordinata del progetto. Questo apre la strada ad eventuali sviluppi e migliorie del prodotto. In particolare, il progetto si predispone all'utilizzo e all'integrazione, completamente o in parte, all'interno di sistemi più grandi, sfruttando dunque il principio del riutilizzo in un dispositivo flessibile che offre delle funzioni utilizzate in diverse tecnologie *smart*.

References

[1] Digilent, *Basys 3 FPGA Board Reference Manual*